Runnable main() for All 20 — Copy, Compile, Test

Each block is FULL working code: includes + function + main(). Paste into a .cpp file, compile with g++, run it. This is how you build real confidence instead of just reading.

1. Maximum Subarray Sum

```
#include <iostream>
using namespace std;
int maxSubArray(int arr[], int n) {
    int maxSum = arr[0], curSum = 0;
    for (int i = 0; i < n; i++) {
        curSum = (curSum + arr[i] > arr[i]) ? curSum + arr[i] : arr[i];
        maxSum = (maxSum > curSum) ? maxSum : curSum;
    }
    return maxSum;
}
int main() {
    int arr[] = {-2,1,-3,4,-1,2,1,-5,4};
    int n = 9;
    cout << "Max subarray sum: " << maxSubArray(arr, n) << endl;  // expect 6
    return 0;
}
```

2. Move Zeroes to End

```
#include <iostream>
using namespace std;
void moveZeroes(int arr[], int n) {
    int writePtr = 0;
    for (int i = 0; i < n; i++) {
        if (arr[i] != 0) {
            int temp = arr[writePtr]; arr[writePtr] = arr[i]; arr[i] = temp;
            writePtr++;
        }
    }
}
int main() {
    int arr[] = {0,1,0,3,12};
    int n = 5;
    moveZeroes(arr, n);
    for (int i = 0; i < n; i++) cout << arr[i] << " ";  // expect 1 3 12 0 0
    cout << endl;
    return 0;
}
```

3. Two Sum (brute force)

```
#include <iostream>
using namespace std;
void twoSum(int arr[], int n, int target, int &i1, int &i2) {
    for (int i = 0; i < n; i++)
        for (int j = i+1; j < n; j++)
            if (arr[i] + arr[j] == target) { i1 = i; i2 = j; return; }
}
int main() {
    int arr[] = {2,7,11,15};
    int n = 4, i1 = -1, i2 = -1;
    twoSum(arr, n, 9, i1, i2);
    cout << "Indices: " << i1 << ", " << i2 << endl;  // expect 0, 1
    return 0;
}
```

4. Best Time to Buy & Sell Stock

```
#include <iostream>
using namespace std;
int maxProfit(int prices[], int n) {
    int minPrice = prices[0], maxProfit = 0;
    for (int i = 0; i < n; i++) {
        if (prices[i] < minPrice) minPrice = prices[i];
        else if (prices[i] - minPrice > maxProfit) maxProfit = prices[i] - minPrice;
    }
```

```
    return maxProfit;
}
int main() {
    int prices[] = {7,1,5,3,6,4};
    int n = 6;
    cout << "Max profit: " << maxProfit(prices, n) << endl;  // expect 5
    return 0;
}
```

5. Product of Array Except Self

```
#include <iostream>
using namespace std;
void productExceptSelf(int arr[], int n, int result[]) {
    result[0] = 1;
    for (int i = 1; i < n; i++) result[i] = result[i-1] * arr[i-1];
    int suffix = 1;
    for (int i = n-1; i >= 0; i--) { result[i] *= suffix; suffix *= arr[i]; }
}
int main() {
    int arr[] = {1,2,3,4};
    int n = 4, result[4];
    productExceptSelf(arr, n, result);
    for (int i = 0; i < n; i++) cout << result[i] << " ";  // expect 24 12 8 6
    cout << endl;
    return 0;
}
```

6. Valid Parentheses

```
#include <iostream>
#include <cstring>
using namespace std;
bool isValid(char s[], int n) {
    char stack[1000]; int top = -1;
    for (int i = 0; i < n; i++) {
        char c = s[i];
        if (c=='('||c=='['||c=='{') stack[++top] = c;
        else {
            if (top == -1) return false;
            char open = stack[top--];
            if ((c==')' && open!='(') || (c==']' && open!='[') || (c=='}' && open!='{'))
                return false;
        }
    }
    return top == -1;
}
int main() {
    char s[] = "{[()]}";
    cout << "Valid: " << (isValid(s, strlen(s)) ? "true" : "false") << endl;  // expect true
    return 0;
}
```

7. Sort a Stack

```
#include <iostream>
using namespace std;
void sortedInsert(int arr[], int &top, int val) {
    if (top == -1 || arr[top] <= val) { arr[++top] = val; return; }
    int temp = arr[top--];
    sortedInsert(arr, top, val);
    arr[++top] = temp;
}
void sortStack(int arr[], int &top) {
    if (top == -1) return;
    int temp = arr[top--];
    sortStack(arr, top);
    sortedInsert(arr, top, temp);
}
int main() {
    int arr[] = {5,2,9,1,7}; int top = 4;  // pretend stack already has these, top index = 4
    sortStack(arr, top);
    for (int i = 0; i <= top; i++) cout << arr[i] << " ";  // expect ascending order
    cout << endl;
    return 0;
}
```

8. Next Greater Element

```
#include <iostream>
using namespace std;
void nextGreater(int arr[], int n, int result[]) {
    int stack[1000], top = -1;
    for (int i = 0; i < n; i++) result[i] = -1;
    for (int i = 0; i < n; i++) {
        while (top != -1 && arr[stack[top]] < arr[i]) { result[stack[top]] = arr[i]; top--; }
        stack[++top] = i;
    }
}
int main() {
    int arr[] = {4,5,2,10,8};
    int n = 5, result[5];
    nextGreater(arr, n, result);
    for (int i = 0; i < n; i++) cout << result[i] << " ";  // expect 5 10 10 -1 -1
    cout << endl;
    return 0;
}
```

9. Min Stack

```
#include <iostream>
using namespace std;
class MinStack {
    int main_[1000], minArr[1000], top;
public:
    MinStack() { top = -1; }
    void push(int val) {
        main_[++top] = val;
        if (top == 0 || val <= minArr[top-1]) minArr[top] = val;
        else minArr[top] = minArr[top-1];
    }
    void pop() { top--; }
    int getTop() { return main_[top]; }
    int getMin() { return minArr[top]; }
};
int main() {
    MinStack st;
    st.push(5); st.push(2); st.push(7);
    cout << "Min: " << st.getMin() << endl;  // expect 2
    cout << "Top: " << st.getTop() << endl;  // expect 7
    return 0;
}
```

10. Reverse Linked List

```
#include <iostream>
using namespace std;
struct Node { int data; Node* next; Node(int v){data=v;next=nullptr;} };
Node* reverseList(Node* head) {
    Node *prev = nullptr, *curr = head;
    while (curr) { Node* next = curr->next; curr->next = prev; prev = curr; curr = next; }
    return prev;
}
int main() {
    Node* head = new Node(1);
    head->next = new Node(2);
    head->next->next = new Node(3);
    Node* rev = reverseList(head);
    while (rev) { cout << rev->data << " "; rev = rev->next; }  // expect 3 2 1
    cout << endl;
    return 0;
}
```

11. Detect Cycle

```
#include <iostream>
using namespace std;
struct Node { int data; Node* next; Node(int v){data=v;next=nullptr;} };
bool hasCycle(Node* head) {
    Node *slow = head, *fast = head;
    while (fast && fast->next) {
        slow = slow->next; fast = fast->next->next;
        if (slow == fast) return true;
```

```
    }
    return false;
}
int main() {
    Node* a = new Node(1); Node* b = new Node(2); Node* c = new Node(3);
    a->next = b; b->next = c; c->next = a;  // cycle back to a
    cout << "Has cycle: " << (hasCycle(a) ? "true" : "false") << endl;  // expect true
    return 0;
}
```

12. Find Middle Node

```
#include <iostream>
using namespace std;
struct Node { int data; Node* next; Node(int v){data=v;next=nullptr;} };
Node* middleNode(Node* head) {
    Node *slow = head, *fast = head;
    while (fast && fast->next) { slow = slow->next; fast = fast->next->next; }
    return slow;
}
int main() {
    Node* head = new Node(1);
    head->next = new Node(2);
    head->next->next = new Node(3);
    head->next->next->next = new Node(4);
    head->next->next->next->next = new Node(5);
    cout << "Middle: " << middleNode(head)->data << endl;  // expect 3
    return 0;
}
```

13. Merge Two Sorted Lists

```
#include <iostream>
using namespace std;
struct Node { int data; Node* next; Node(int v){data=v;next=nullptr;} };
Node* mergeTwoLists(Node* l1, Node* l2) {
    Node dummy(0); Node* tail = &dummy;
    while (l1 && l2) {
        if (l1->data <= l2->data) { tail->next = l1; l1 = l1->next; }
        else { tail->next = l2; l2 = l2->next; }
        tail = tail->next;
    }
    tail->next = l1 ? l1 : l2;
    return dummy.next;
}
int main() {
    Node* l1 = new Node(1); l1->next = new Node(3); l1->next->next = new Node(5);
    Node* l2 = new Node(2); l2->next = new Node(4); l2->next->next = new Node(6);
    Node* merged = mergeTwoLists(l1, l2);
    while (merged) { cout << merged->data << " "; merged = merged->next; }  // expect 1 2 3 4 5 6
    cout << endl;
    return 0;
}
```

14. Nth Node from End

```
#include <iostream>
using namespace std;
struct Node { int data; Node* next; Node(int v){data=v;next=nullptr;} };
Node* nthFromEnd(Node* head, int n) {
    Node *fast = head, *slow = head;
    for (int i = 0; i < n; i++) fast = fast->next;
    if (!fast) return head;
    while (fast->next) { fast = fast->next; slow = slow->next; }
    return slow->next;
}
int main() {
    Node* head = new Node(1);
    head->next = new Node(2);
    head->next->next = new Node(3);
    head->next->next->next = new Node(4);
    head->next->next->next->next = new Node(5);
    cout << "2nd from end: " << nthFromEnd(head, 2)->data << endl;  // expect 4
    return 0;
}
```

15. Maximum Path Sum

```
#include <iostream>
using namespace std;
struct TreeNode { int val; TreeNode *left,*right; TreeNode(int v){val=v;left=right=nullptr;} };
int ans = -1000000;
int dfsMax(TreeNode* root) {
    if (!root) return 0;
    int left = dfsMax(root->left); if (left < 0) left = 0;
    int right = dfsMax(root->right); if (right < 0) right = 0;
    int through = root->val + left + right;
    if (through > ans) ans = through;
    return root->val + (left > right ? left : right);
}
int main() {
    TreeNode* root = new TreeNode(-10);
    root->left = new TreeNode(9);
    root->right = new TreeNode(20);
    root->right->left = new TreeNode(15);
    root->right->right = new TreeNode(7);
    dfsMax(root);
    cout << "Max path sum: " << ans << endl;  // expect 42
    return 0;
}
```

16. Level Order Traversal

```
#include <iostream>
using namespace std;
struct TreeNode { int val; TreeNode *left,*right; TreeNode(int v){val=v;left=right=nullptr;} };
void levelOrder(TreeNode* root) {
    if (!root) return;
    TreeNode* queue[1000]; int front = 0, back = 0;
    queue[back++] = root;
    while (front < back) {
        int levelSize = back - front;
        for (int i = 0; i < levelSize; i++) {
            TreeNode* node = queue[front++];
            cout << node->val << " ";
            if (node->left) queue[back++] = node->left;
            if (node->right) queue[back++] = node->right;
        }
        cout << endl;
    }
}
int main() {
    TreeNode* root = new TreeNode(3);
    root->left = new TreeNode(9);
    root->right = new TreeNode(20);
    root->right->left = new TreeNode(15);
    root->right->right = new TreeNode(7);
    levelOrder(root);  // expect: 3 / 9 20 / 15 7
    return 0;
}
```

17. Height of Binary Tree

```
#include <iostream>
using namespace std;
struct TreeNode { int val; TreeNode *left,*right; TreeNode(int v){val=v;left=right=nullptr;} };
int height(TreeNode* root) {
    if (!root) return 0;
    int l = height(root->left), r = height(root->right);
    return 1 + (l > r ? l : r);
}
int main() {
    TreeNode* root = new TreeNode(1);
    root->left = new TreeNode(2);
    root->left->left = new TreeNode(3);
    cout << "Height: " << height(root) << endl;  // expect 3
    return 0;
}
```

18. Diameter of Tree

```
#include <iostream>
```

```
using namespace std;
struct TreeNode { int val; TreeNode *left,*right; TreeNode(int v){val=v;left=right=nullptr;} };
int diameter = 0;
int heightDia(TreeNode* root) {
    if (!root) return 0;
    int left = heightDia(root->left), right = heightDia(root->right);
    if (left + right > diameter) diameter = left + right;
    return 1 + (left > right ? left : right);
}
int main() {
    TreeNode* root = new TreeNode(1);
    root->left = new TreeNode(2);
    root->right = new TreeNode(3);
    root->left->left = new TreeNode(4);
    root->left->right = new TreeNode(5);
    heightDia(root);
    cout << "Diameter: " << diameter << endl;  // expect 3
    return 0;
}
```

19. Number of Islands

```
#include <iostream>
using namespace std;
void dfs(char grid[][5], int rows, int cols, int r, int c) {
    if (r < 0 || c < 0 || r >= rows || c >= cols || grid[r][c] == '0') return;
    grid[r][c] = '0';
    dfs(grid, rows, cols, r+1, c); dfs(grid, rows, cols, r-1, c);
    dfs(grid, rows, cols, r, c+1); dfs(grid, rows, cols, r, c-1);
}
int numIslands(char grid[][5], int rows, int cols) {
    int count = 0;
    for (int i = 0; i < rows; i++)
        for (int j = 0; j < cols; j++)
            if (grid[i][j] == '1') { count++; dfs(grid, rows, cols, i, j); }
    return count;
}
int main() {
    char grid[2][5] = {
        {'1','1','0','1','1'},
        {'1','1','0','0','1'}
    };
    cout << "Islands: " << numIslands(grid, 2, 5) << endl;  // expect 2
    return 0;
}
```

20. Median from Data Stream

```
#include <iostream>
using namespace std;
int data[10000], count_ = 0;
void addNum(int num) {
    int i = count_ - 1;
    while (i >= 0 && data[i] > num) { data[i+1] = data[i]; i--; }
    data[i+1] = num;
    count_++;
}
double findMedian() {
    if (count_ % 2 == 1) return data[count_/2];
    return (data[count_/2 - 1] + data[count_/2]) / 2.0;
}
int main() {
    addNum(1); addNum(2);
    cout << "Median: " << findMedian() << endl;   // expect 1.5
    addNum(3);
    cout << "Median: " << findMedian() << endl;   // expect 2
    return 0;
}
```

How to actually run these

```
# save any block as solution.cpp, then in terminal:
g++ solution.cpp -o solution
```

```
./solution

# or use any online compiler: onlinegdb.com, replit.com, godbolt.org
```